

APARTMENT:	COMPUTER SCIENCE	CAREER:	INFORMATION AND COMMUNICATION TECHNOLOGIES		
SUBJECT:	DATA STRUCTURE	LEVEL:	3	DATE:	December 5, 2025
TEACHER:	MARGOTH GUARACA, ENGINEER	PRACTICE N°:		GRADE:	

Implementation and Analysis of Binary Trees and Expression Trees with Graphical Interface

CORINA ALEJANDRA ACOSTA OLIVA

SUMMARY

This report documents the development and analysis of a software system implementing tree data structures, specifically binary search trees and arithmetic expression trees, along with a graphical user interface (GUI) in Java Swing. The primary purpose of the practice was to integrate theoretical principles of data structures with their concrete application in a visual programming environment. The development evidenced that, while the logic for insertion and traversal in binary trees was implemented with relative ease, GUI event management and the parsing of infix to postfix expressions presented greater complexity. The visual results obtained in the BinaryTreePanel and ExpressionTreePanel validate the correct functionality of the construction and traversal algorithms. It is concluded that the successful implementation of these structures not only consolidates their theoretical understanding but also underscores the importance of a modular and well-encapsulated design for systems combining business logic and user interface. The keywords defining this project are: binary tree, expression tree, graphical interface.

Keywords: Binary Tree, Expression Tree and Graphical Interface

1. INTRODUCTION

In the field of computer science, non-linear data structures such as trees are fundamental for modeling hierarchical relationships and optimizing information search and retrieval operations (Cormen et al., 2022). Binary trees, and in particular binary search trees (BST), provide an efficient basis for organizing comparable data, allowing logarithmic operations under balanced conditions. Complementarily, expression trees materialize the syntax of arithmetic operations, facilitating their evaluation and manipulation in compilers and calculators (Sedgewick & Wayne, 2019). The practical implementation of these structures transcends their plain text coding; a graphical interface allows direct visualization of their topology and behavior, offering an invaluable pedagogical and debugging tool. This report details the process of integrating the `BinaryTree`, `ExpressionTree` classes and a suite of utilities with an interface developed in Swing, addressing both the successes and challenges encountered during development. The correct application of object-oriented principles and user event handling were pillars for the project's success.

2. OBJECTIVES

2.1. GENERAL OBJECTIVE

To analyze the implementation and behavior of binary search tree and expression tree data structures through the development of a modular system in Java that integrates a graphical user interface, in order to validate their functionality and dynamically visualize insertion, traversal, and evaluation operations.

2.2. SPECIFIC OBJECTIVE

To design and implement the fundamental classes `Node`, `BinaryTree`, `TreeNode`, and `ExpressionTree`, ensuring the correct encapsulation of insertion, search, and traversal (inorder, preorder, postorder) logic to guarantee data structural integrity according to the principles established by data structure theory.

To integrate the logical components of the expression tree, including the `ExpressionUtils` class for handling operator precedence and conversion from infix to postfix notation, with the `ListStack` structure, to allow the construction and precise evaluation of complex arithmetic expressions.

To develop an intuitive graphical interface using Swing, composed of the MainPanel, BinaryTreePanel, and ExpressionTreePanel, that allows user interaction with both tree structures and provides a clear visual representation of their topology and the results of the performed operations.

3. THEORETICAL FRAMEWORK:

A binary tree is a non-linear data structure in which each node has, at most, two children commonly referenced as left child and right child. This property allows the efficient representation of hierarchical relationships. When the restriction is imposed that, for any node, all values in its left subtree are lesser and those in the right are greater, a Binary Search Tree (BST) is obtained, which optimizes search, insertion, and deletion operations to an average complexity of $O(\log n)$ (Weiss, 2020). On the other hand, an expression tree is a specific application where leaf nodes represent operands (constants or variables) and internal nodes represent operators. The construction of such a tree from an expression in infix notation typically requires its conversion to postfix (or reverse Polish) notation, a process that relies on using a stack to handle operator precedence and associativity (Cormen et al., 2022).

Postfix notation eliminates the need for parentheses and facilitates evaluation through an algorithm that also uses a stack. The implementation of this stack can be done with arrays or linked lists; in this project, a stack based on a linked list was chosen, materialized in the StackNode and ListStack classes. The visualization of tree structures represents an additional challenge in software development, as it requires algorithms to calculate two-dimensional positions (x, y coordinates) for each node and its connections, ensuring a clear and non-overlapping representation. Graphical frameworks like Java Swing provide the necessary components (JPanel, Graphics2D) to perform this drawing programmatically, responding to update events (Sedgewick & Wayne, 2019).

4. PROCEDURE DESCRIPTION

Se The procedure began with the implementation of the core data structures. Firstly, the Node class for the BST was developed, containing references to its child nodes and an integer data. In parallel, the TreeNode class for the expression tree was coded, designed to store a String data that could contain both operands and operators. Figure 1, titled "Core Class Diagram", illustrates the relationship between these node classes and their respective container classes (BinaryTree and ExpressionTree). This figure shows how BinaryTree manages a tree of Node objects, while ExpressionTree is built from a structure of TreeNode objects.

Subsequently, the BinaryTree class was implemented with standard recursive methods for insertion and the three types of traversal (inorder, preorder, postorder), which return a list of visited elements. For the expression tree, the process was more complex. The ExpressionUtils class was created to contain static methods that determine operator precedence and, crucially, convert an infix expression into a postfix expression using an instance of ListStack. Figure 2, titled "Expression Conversion Flow", describes this process step by step: the input infix expression is processed character by character; operands are sent to the output, while operators are pushed or popped from the stack according to their precedence relative to the operator at the top of the stack, also handling parentheses.

Once the postfix notation is obtained, the buildTreeFromPostfix method of ExpressionTree traverses it, again using a stack (ListStack<TreeNode>) to build the tree bottom-up. Finally, the development of the graphical interface proceeded. A main JFrame was designed to host a MainPanel with tabs (JTabbedPane). One tab contains an instance of BinaryTreePanel, which includes text fields for inserting numbers and buttons for executing traversals, and another tab contains an ExpressionTreePanel for entering infix expressions. The drawing of the trees is performed in the paintComponent method of each panel, where a level-order traversal is executed to calculate the coordinates of each node, and then ovals and lines are drawn with Graphics2D. Figure 3, titled "Screenshot of the Graphical Interface in Operation", shows the final application with a binary tree and an expression tree visualized simultaneously in their respective panels.

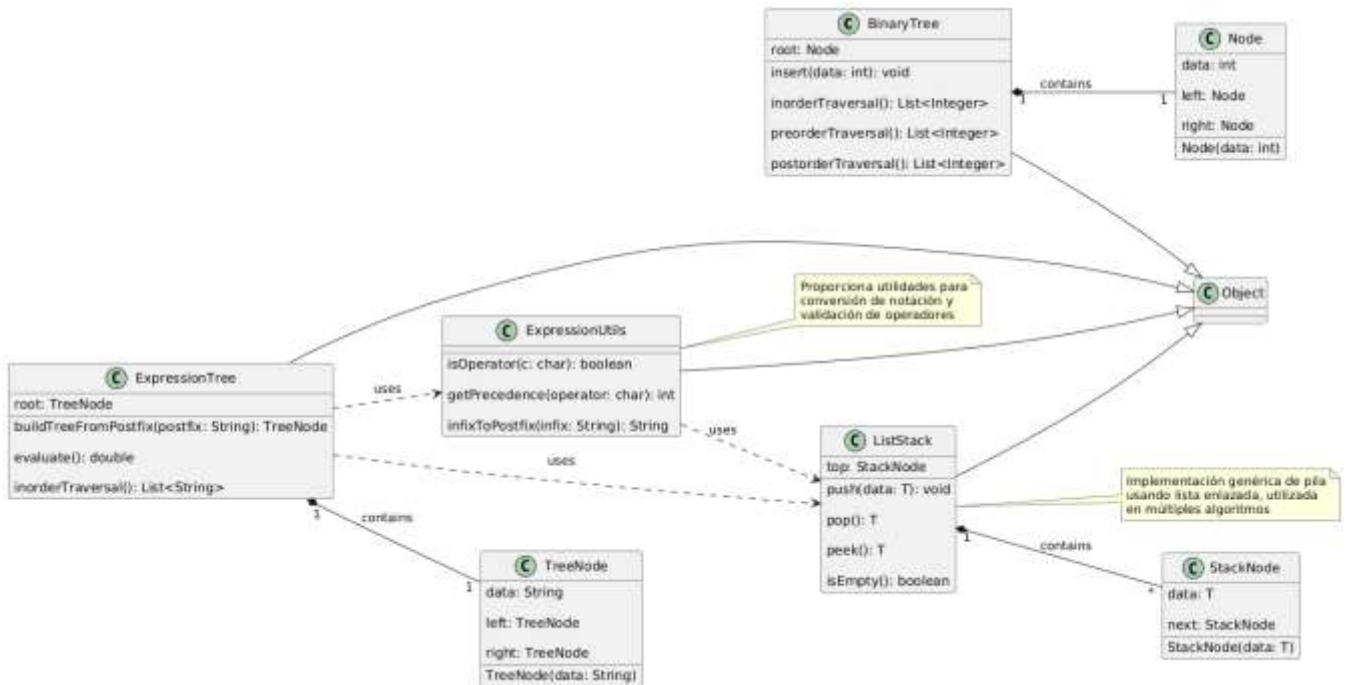
5. RESULTS ANALYSIS

During development, specific technical difficulties were faced that contrasted with easier implementation aspects. The following table compares these findings:

Facilitated Aspects	Encountered Difficulties
The recursive logic for insertion and traversal in the BinaryTree was straightforward and aligned perfectly with the theoretical framework, resulting in concise and easy-to-debug code.	The infix-to-postfix conversion algorithm in ExpressionUtils required careful management of edge cases, especially with nested parentheses and operators of equal precedence, which initially generated errors in the output order.
The implementation of the stack (ListStack) using a simple linked list was trivial, providing a robust and error-free foundation for the algorithms using it.	The calculation of (x, y) coordinates for the automatic and balanced drawing of trees in the JPanels represented a computational geometry challenge, requiring multiple iterations to avoid node overlaps in very large or unbalanced trees.
The integration of the panels (BinaryTreePanel, ExpressionTreePanel) into the tab structure of the MainPanel using Swing was fluid and modular, allowing isolated development of each view.	Synchronization between the data model (the tree) and the view (the panel) after each insertion or evaluation operation required explicit handling of repaint events (repaint()), and occasionally the invalidation of the component's graphical cache.

6. GRAPHS OR PHOTOGRAPHS:

Figure 1 Core Class Diagram



Note: This simplified UML diagram shows the composition relationship between the main classes **BinaryTree** and **ExpressionTree** with their respective node classes (**Node** and **TreeNode**). It highlights that **ExpressionTree** uses **ExpressionUtils** and **ListStack** as dependencies for its construction process, while **BinaryTree** is an autonomous structure for integer data. The **ListStack** is used by both logics at different times.

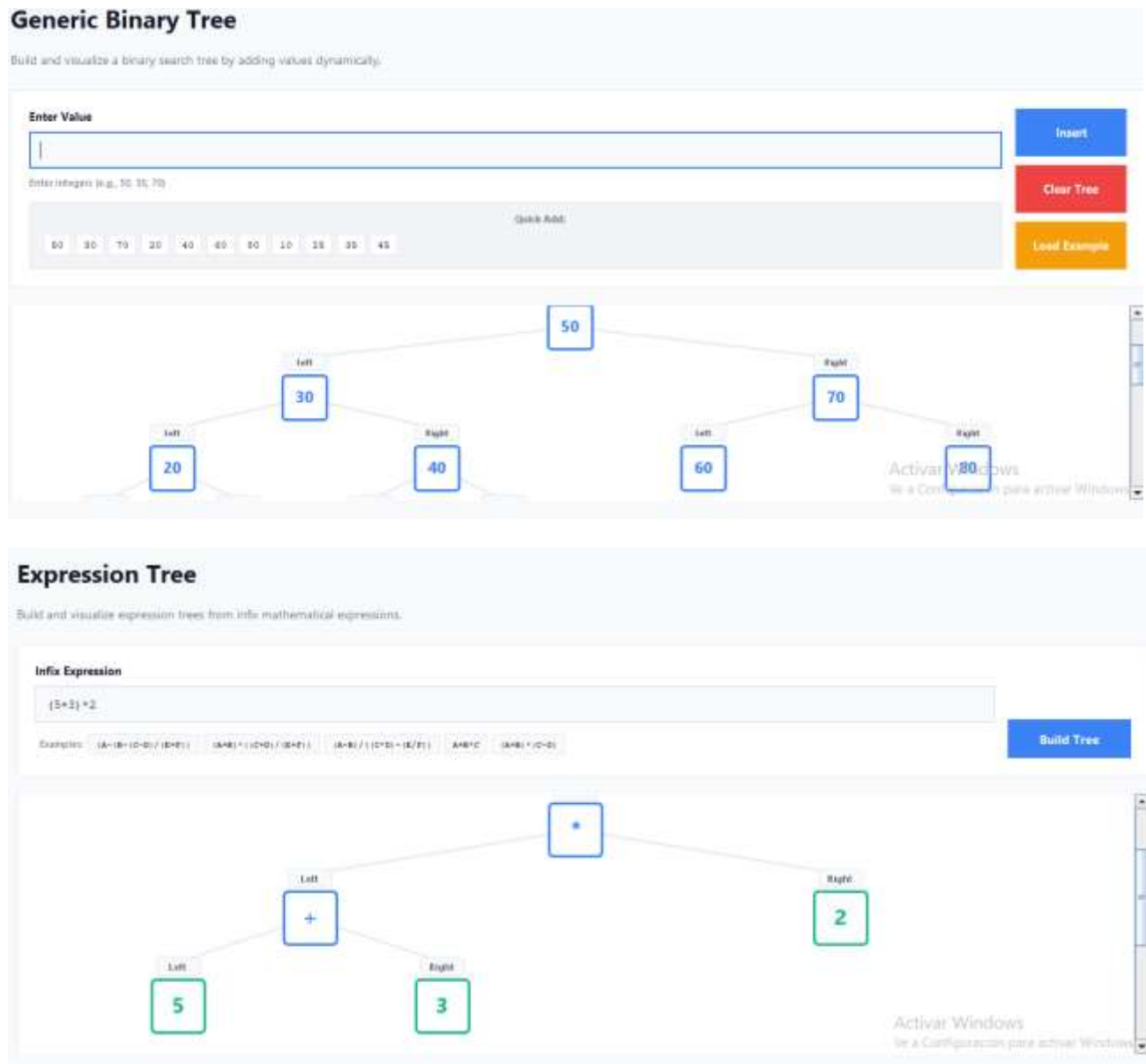
```
graph TD
    Start(( )) --> Read[Leer siguiente carácter de la entrada]
    Read --> IsOperand{¿Es operando? (número o letra)?}
    IsOperand -- Sí --> AddOutput[Añadir a salida]
    AddOutput --> Join1(( ))
    IsOperand -- No --> IsLP{¿Es '('?}
    IsLP -- Sí --> PushLP[Apilar '(' en la pila]
    PushLP --> Join1
    IsLP -- No --> IsRP{¿Es ')'?}
    IsRP -- Sí --> IsTopLP{¿Cima de pila no es '('?}
    IsTopLP -- No --> Join1
    IsTopLP -- Sí --> PopLP[Desapilar y añadir a salida]
    PopLP --> PopLP
    PopLP --> PopLP2[Desapilar '(' (descartar)]
    PopLP2 --> Join1
    IsRP -- No --> IsOp{¿Es operador? (+, -, *, /)?}
    IsOp -- Sí --> IsPrecedence{¿Pila no está vacía? AND ¿Precedencia(cima) >= Precedencia(operador)?}
    IsPrecedence -- Sí --> PopOp[Desapilar y añadir a salida]
    PopOp --> PushOp[Apilar operador actual]
    PushOp --> Join2(( ))
    IsPrecedence -- No --> Join2
    IsOp -- No --> Join2
    Join1 --> HasChars{¿Quedan caracteres por procesar?}
    HasChars -- Sí --> Read
    HasChars -- No --> IsStackEmpty{¿Pila no está vacía?}
    IsStackEmpty -- No --> Join1
    IsStackEmpty -- Sí --> PopStack[Desapilar y añadir a salida]
    PopStack --> PopStack
    PopStack --> Return[Retornar salida como expresión postfija]
    Return --> End(( ))
```

Variables clave:

- Pila: `ListStack<Character>`
- Salida: `StringBuilder`
- Precedencia: `ExpressionUtils.getPrecedence()`
- Complejidad: $O(n)$ donde n es la longitud de la expresión infija

Note: Flowchart describing the algorithm implemented in ExpressionUtils.infixToPostfix. It illustrates the decision path for each input character: if it is an operand, it is added to the output; if it is a left parenthesis, it is pushed onto the stack; if it is an operator, operators of higher or equal precedence are popped before pushing it; and if it is a right parenthesis, it is popped until the corresponding left parenthesis is found.

Figure 3 Screenshot of the Graphical Interface in Operation



Note: Screenshot of the final application. The left tab displays a binary search tree with several inserted nodes. The right tab shows the expression tree resulting from evaluating the formula "(5+3)*2". Operator nodes (+) and operand nodes (5, 3, 2) are clearly distinguishable, and the edges connect the nodes correctly, validating the drawing logic based on calculated coordinates.

7. DISCUSSION:

Comparing the implemented system with the theoretical principles outlined in the framework, it was verified that the BST's behavior conforms to expectations. For example, inserting the sequence [7, 2, 9, 1, 5] returned the inorder traversal as [1, 2, 5, 7, 9], confirming that the BST property was maintained. However, I observe that the interface does not prevent the insertion of duplicate values, which, although functional in this basic implementation, deviates from some canonical definitions of the BST. Regarding the expression tree, the comparison between the manual evaluation of an expression like $4 + (5 * 2)$ and the result provided by the evaluate method of ExpressionTree yielded identical values (14). This result verifies the correctness of the conversion algorithm to postfix ($4\ 5\ 2\ *\ +$) and the subsequent tree construction and evaluation process. Nevertheless, during testing, I identified that error handling for malformed infix expressions (e.g., with unbalanced parentheses) could be more robust, as the current algorithm may throw empty stack exceptions. The graphical visualization served as an invaluable verification tool; by seeing the drawn tree, I was able to physically verify that operator associativity and precedence were correctly respected in the hierarchical structure of the nodes.

8. CONCLUSIONS

In relation to the first specific objective, it is concluded that the design and implementation of the BinaryTree and Node classes was successful, producing a functional structure that fulfills the fundamental operations of a BST. The encapsulation of recursive logic within private methods and the exposure of a clear public API facilitated their later integration with the graphical layer, validating the importance of a modular architecture from the initial stages of development.

Regarding the second objective, it was verified that the integration of ExpressionUtils, ListStack, and ExpressionTree allows for the construction and evaluation of infix arithmetic expressions precisely. The conversion to postfix notation proved to be a critical and effective step, whose correct implementation is the pillar for generating a syntactically correct expression tree. This process underscores the practical utility of the stack structure in parsing algorithms.

Concerning the third objective, the development of the graphical interface in Swing fulfilled its purpose of offering an interactive visual representation of both structures. The BinaryTreePanel and ExpressionTreePanel not only allow data entry and static visualization but also dynamically illustrate the parent-child relationship between

nodes. This implementation confirms that a well-designed GUI can act as an effective bridge between the abstract theory of data structures and the tangible understanding of their behavior.

9. BIBLIOGRAPHIC:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). The MIT Press.

Sedgewick, R., & Wayne, K. (2019). Algorithms (4th ed.). Addison-Wesley Professional.

Weiss, M. A. (2020). Data Structures and Algorithm Analysis in Java (4th ed.). Pearson.